

400 HTTP Status Code in ASP.NET Core Web API

Back to: [ASP.NET Core Web API Tutorials](#)

400 HTTP Status Code in ASP.NET Core Web API

In this article, I will discuss **how to Return 400 Bad Request HTTP Status Code from the ASP.NET Core Web API** Controller Action method with Examples. Please read our previous article on how to Return [304 HTTP Status Code in ASP.NET Core Web API](#) with Examples.

400 HTTP Status Code

The 400 HTTP Status Code, also known as “Bad Request,” indicates that the request sent to the server by the client was somehow incorrect or corrupted, and the Server cannot process the request. This can happen for various reasons, including:

- **Invalid URL:** The URL might be incorrect or improperly formatted, or contain illegal characters.
- **Invalid Query Parameters:** The query string parameters might be missing or incorrectly formatted.
- **Invalid Headers:** Headers sent in the request may be invalid or missing.
- **Large Payload:** The request payload might be too large for the server to handle.
- **Invalid Cookies:** There may be issues with the cookies (Corrupted or invalid cookies) being sent with the request.
- **Invalid Request Payload:** The body of the request might not be in the expected format or contain invalid data.
- **Invalid Content Type:** The Content-Type header might be incorrect or not supported by the server.

How to Resolve a 400 Bad Request Error:

When developing web applications, i.e., client applications, it's important to handle 400 errors. To troubleshoot and fix a 400 Bad Request Error, we should do the following:

- **Check the URL:** Ensure that the URL is correct and properly formatted.
- **Validate Query String Parameters:** Confirm that all query parameters are correctly encoded and valid.
- **Review Headers:** Make sure all required headers are correctly set and valid.
- **Inspect the Request Payload:** Ensure the request payload is not too large and follows the expected format.
- **Clear Cookies:** Try clearing cookies if they might be causing the issue.
- **Verify Content-Type:** Ensure the Content-Type header matches the payload format.

Note: When a server returns a 400 status, it should provide more information about the error in the response body, helping the client to understand what specifically went wrong and how to fix it. It's a way for the server to

communicate to the client that the issue lies with the incoming request rather than with the server itself.

How to Return 400 HTTP Status Code in ASP.NET Core Web API?

To return a 400 HTTP status code in an ASP.NET Core Web API, you can use various built-in methods that the framework provides. These methods make it straightforward to send specific HTTP responses back to the client, ensuring that you communicate the right status clearly and efficiently. The following are some of the ways to return a 400 Bad Request response in ASP.NET Core Web API:

Using BadRequest Method

The `BadRequest()` method is part of the `ControllerBase` class, which the API controllers typically inherit from. This method is useful for quickly returning a 400 status code when the input validation fails or the request data is incorrect.

Let us understand this with an example. First, create the following `Employee` class within the `Models` folder. By default, all the properties of the following `Employee` model are mandatory or, you can say, required. If a property is a nullable type, then it is optional. Else, it is mandatory. Here, we are not creating any property as a nullable type by putting the question mark.

```
namespace ReturnTypeAndStatusCodes.Models
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Gender { get; set; }
        public string City { get; set; }
        public int Age { get; set; }
        public string Department { get; set; }
    }
}
```

Next, modify the `Employee` Controller as follows. Here, you can see that within the `CreateEmployee` method, we are checking the Model state. If it is not valid, we are returning a 400 Bad Request HTTP Status Code to the client indicating the Request data is Invalid for creating a new `Employee`.

```
using Microsoft.AspNetCore.Mvc;
using ReturnTypeAndStatusCodes.Models;

namespace ReturnTypeAndStatusCodes.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
```

```

public class EmployeeController : ControllerBase
{
    [HttpPost]
    public IActionResult CreateEmployee([FromBody] Employee employee)
    {
        if (!ModelState.IsValid)
        {
            // Returns a 400 Status Code with model state errors
            return BadRequest(ModelState);
        }

        // Process the request...

        // Returns a 200 Status Code indicating success
        return Ok();
    }
}

```

Testing the API:

URL: <https://localhost:7094/api/Employee>

Method: POST

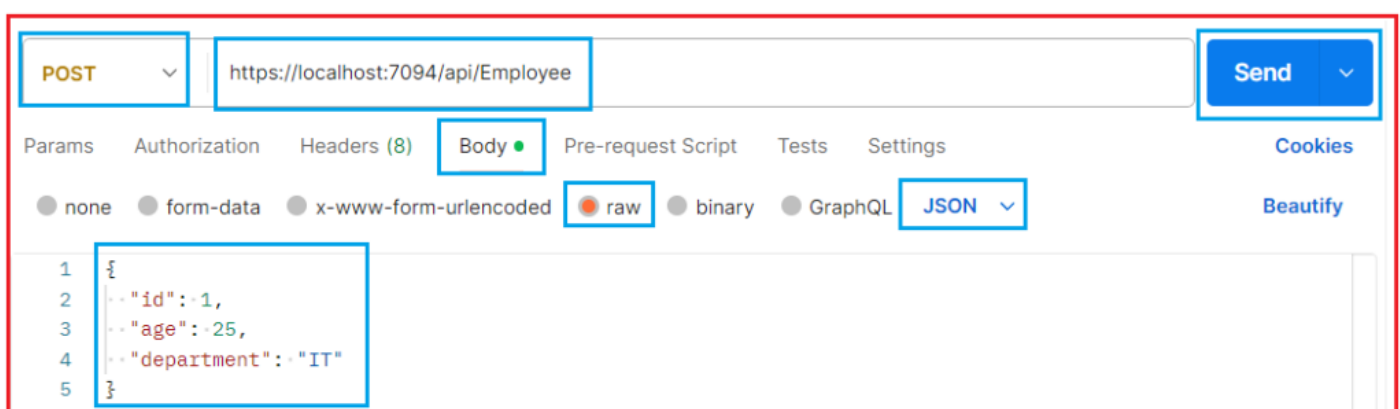
Request Body: I am not passing the required field values for Name, City, and Gender in the request body.

```

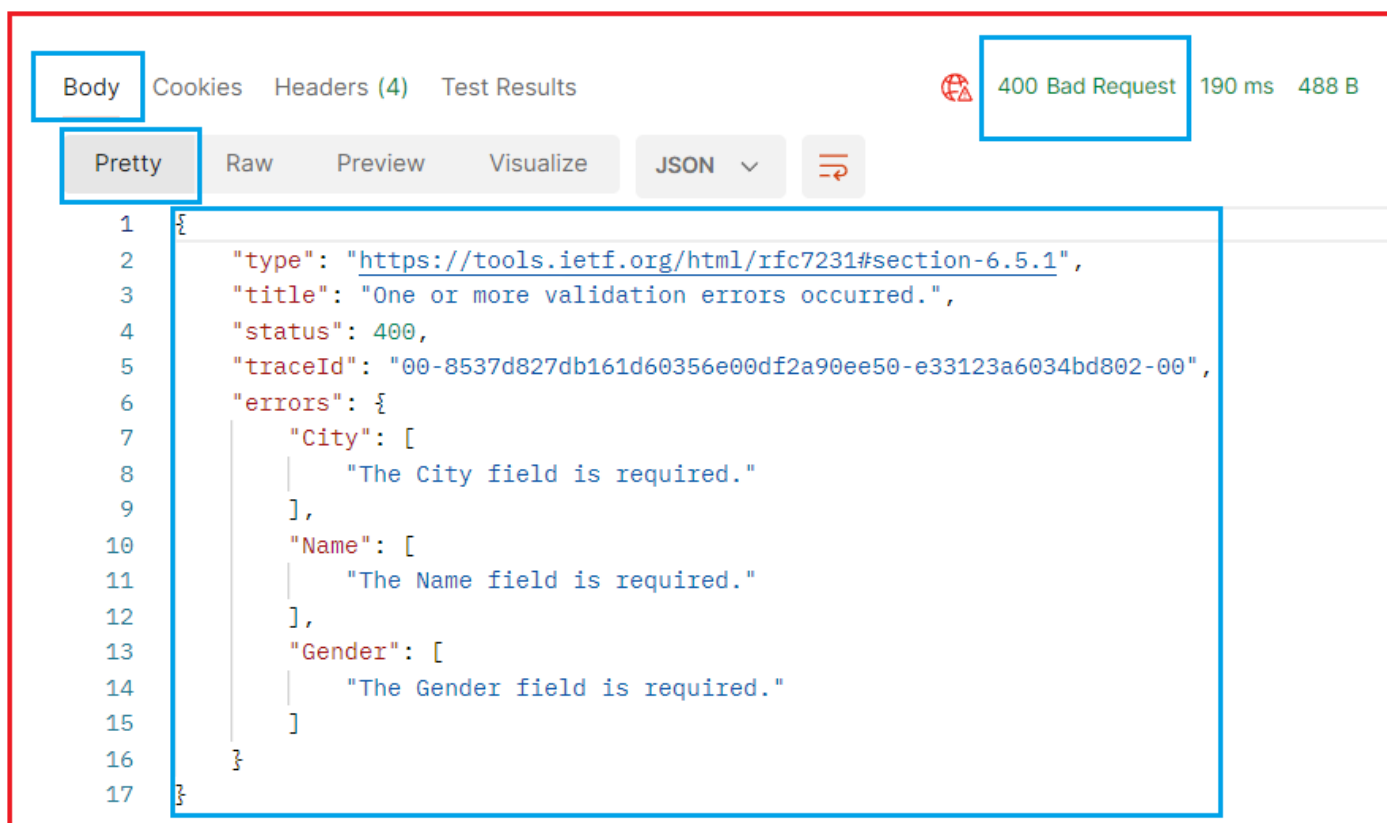
{
  "id": 1,
  "age": 25,
  "department": "IT"
}

```

So, open Postman, provide the above details, and click on the Send button, as shown in the image below. This will send a Post request to the server. Please change the port number where your application is running.



Once you click on the Send button, you will get the following response.



As you can see in the above image, the response body shows all the details about an error with the status code 400. When we encounter a **400 Bad Request** response, it typically includes several pieces of information to help diagnose the issue. They are as follows:

- **type:** This is a URI that points to a human-readable document describing the specific error type. In this case, the URI links to Section 15.5.1 of RFC 9110, which details the appropriate use of HTTP status codes and problem details in APIs. For more details, copy the URL and open it on your web browser.
- **title:** It provides a brief, human-readable summary of the nature of the problem. Here, it gives "One or more validation errors occurred." This indicates that the request failed due to validation errors in the input data.
- **status:** The HTTP status code associated with this occurrence of the problem. Here, it is 400, which is the status code for "Bad Request." This code is used when the request cannot be processed due to client error, which is typically invalid request data.
- **errors:** Contains the fields that failed validation along with a list of relevant error messages for each field. Here, it gives the following error messages:
 1. **City:** The error message "The City field is required." indicates that the City field was not provided but is expected.
 2. **Name:** Similar to City, the Name field was expected but not provided.
 3. **Gender:** Again, this indicates the Gender field was required but missing in the input.
- **traceId:** This is a unique identifier for the request, useful for tracing and debugging purposes. It helps developers and support teams correlate the request with server logs and troubleshoot any issues. ASP.NET Core automatically generates and includes a TraceId in responses if logging and diagnostics are correctly configured in the application.

How It's Generated

This response is automatically generated by ASP.NET Core when model validation fails. When an action method in a controller is decorated with the `[ApiController]` attribute, the framework automatically performs model validation before executing the action. If the validation fails, it returns a `ValidationProblemDetails` object with the structure shown above. This is handled automatically by the Framework before executing the action method, so **ModelState.IsValid** checking is not required within the action method. That means the following code will work as expected, and you will also get the same `BadRequest` error:

```
using Microsoft.AspNetCore.Mvc;
using ReturnTypeAndStatusCodes.Models;

namespace ReturnTypeAndStatusCodes.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpPost]
        public IActionResult CreateEmployee([FromBody] Employee employee)
        {
            // Process the request...

            // Returns a 200 Status Code indicating success
            return Ok();
        }
    }
}
```

Returning BadRequestObjectResult Directly

If you need to return a specific error message or details about what went wrong with the request, you can use `BadRequestObjectResult`. Please comment on the `ApiController` attribute to work this correctly. Otherwise, before executing the action method, `ApiController` will check the model, and if it is found invalid, it will return the default `BadRequest` error response. So, please modify the `Employee Controller` as follows:

```
using Microsoft.AspNetCore.Mvc;
using ReturnTypeAndStatusCodes.Models;

namespace ReturnTypeAndStatusCodes.Controllers
{
    [Route("api/[controller]")]
    //[ApiController]
    public class EmployeeController : ControllerBase
    {
```

```

[HttpPost]
public IActionResult CreateEmployee([FromBody] Employee employee)
{
    // Process the request...
    if (!ModelState.IsValid)
    {
        //var errorResponse = new { error = "Invalid Request Data",
        details = ModelState };
        var errorResponse = new { error = "Invalid Request Data"};
        return new BadRequestObjectResult(errorResponse); // Returns 400
        with a custom object
    }
    // Returns a 200 Status Code indicating success
    return Ok();
}
}
}

```

ProblemDetails for API Errors in ASP.NET Core

The ProblemDetails class in ASP.NET Core Web API is designed to encapsulate error details in a structured format, making it easier for API consumers to understand what went wrong when an error occurred. If you go to the definition of ProblemDetails class, you will find the following signature:

```

public class ProblemDetails
{
    ... public string? Type ...
    ... public string? Title ...
    ... public int? Status ...
    ... public string? Detail ...
    ... public string? Instance ...
    ... public IDictionary<string, object?> Extensions { get; set; }
}

```

Let us understand each property in the ProblemDetails class:

Type:

Provides a URI that identifies the type of the problem. It provides a reference to a specific problem type for further information. This is an optional property and can be used to link to a page or documentation that provides more details about the specific type of error. You might use a URI like <https://tools.ietf.org/html/rfc7231#section-6.5.1>, which provides information about specific errors such as “400 Bad Request”.

Title:

A short, human-readable summary of the problem type. This provides a brief explanation of the problem. This property is typically a short, plain text description of the problem. It's meant to be readable by end-users and help them understand what went wrong, such as "Invalid Request Parameters" or "Authentication Failed."

Status:

The HTTP status code generated by the server for this occurrence of the problem. This property holds the HTTP status code that corresponds to the problem, like 400, 404, or 500. It helps clients understand what HTTP error code to expect, such as 404 for "Not Found," 400 for "Bad Request," or 500 for "Internal Server Error."

Detail:

A human-readable explanation that is specific to this occurrence of the problem. It should provide more details about what went wrong than the title. This property provides additional information that can be useful for debugging or for informing the user about the specifics of the problem. It is optional and can be used to give context or specific error messages. For example, "The field 'username' is required but was not provided."

Instance:

A URI reference that identifies the specific occurrence of the problem. It is useful for tracking and diagnosing the problem in a particular context. This property can provide a link or identifier related to the specific instance of the problem. It helps clients and developers track or reference the specific occurrence of the issue. For example, the path of a log file such as <https://example.com/logs/12345>, where the clients or developers can see the detailed error log information.

Extensions:

This allows for extending the problem details object with additional application-specific properties and adding custom properties that are not covered by the standard properties. It is a dictionary that can hold any number of custom key-value pairs. For example, adding a `retryAfter` key that indicates how many seconds to wait before making another request.

Example to Understand Problem Details in ASP.NET Core Web API:

The `ProblemDetails` object is typically used in API controllers to create an instance of it and populate its properties based on the specific error scenario. This object can then be returned with the appropriate HTTP status code. For instance, in an API method, you might return a 404 error. To better understand the `ProblemDetails` class in ASP.NET Core Web API, please modify the `Employee Controller` as follows. Please comment on the `ApiController` attribute to make this work correctly.

```
using Microsoft.AspNetCore.Mvc;
```

```

using ReturnTypesAndStatusCodes.Models;

namespace ReturnTypesAndStatusCodes.Controllers
{
    [Route("api/[controller]")]
    // Uncomment this attribute to enable automatic model validation and other
    features
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        // Define a POST endpoint for creating an employee
        [HttpPost]
        public IActionResult CreateEmployee([FromBody] Employee employee)
        {
            // Check if the model state is valid
            if (!ModelState.IsValid)
            {
                // Create a dictionary to hold the details of each validation
                error
                var errors = ModelState
                    .Where(e => e.Value?.Errors.Count > 0) // Filter model state
                    entries that have errors
                    .ToDictionary(
                        kvp => kvp.Key, // Use the property name as the key
                        kvp => kvp.Value?.Errors.Select(e =>
e.ErrorMessage).ToArray() // Use the error messages as the value
                    );

                // Create a ProblemDetails object to hold the error response
                details
                var problemDetails = new ProblemDetails()
                {
                    Title = "Validation Errors Occurred.", // Short description
                    of the problem
                    Detail = "See the errors property for details", // Detailed
                    description of the problem
                    Type = "https://tools.ietf.org/html/rfc7231#section-6.5.1",
                    // URI reference for the problem type
                    Status = StatusCodes.Status400BadRequest, // HTTP status code
                    for the response
                    Instance = HttpContext.Request.Path, // The path to the
                    request that generated the error
                    Extensions = new Dictionary<string, object?> // Additional
                    custom data
                    {
                        { "Retry", "Please Retry After 30 Minutes" }, // Custom
                        retry message

```



```

        { "errors", errors } // Include the detailed validation
errors

    }

};

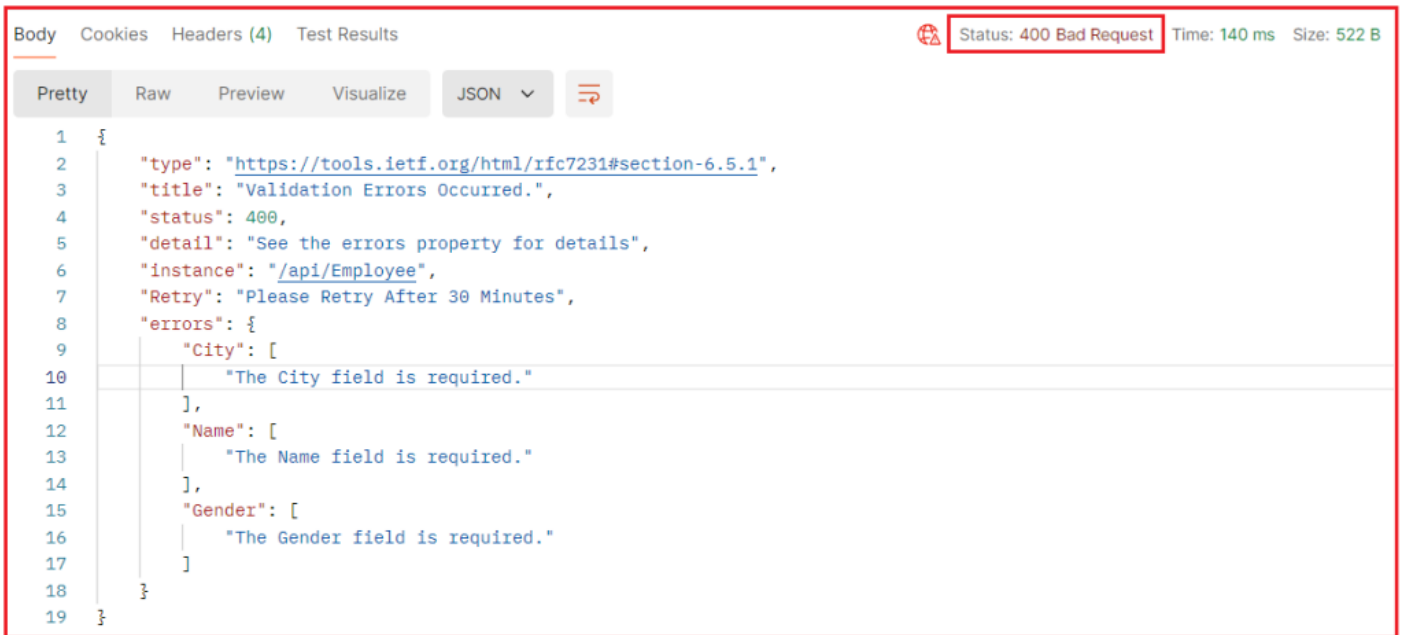
// Return a 400 Bad Request response with the problem details
return BadRequest(problemDetails);
}

// Process the request (e.g., save the employee data to the database)
// ...

// Return a 200 OK response indicating success
return Ok();
}
}
}

```

Response:



The screenshot shows a web browser's developer tools interface. The 'Body' tab is selected, displaying a JSON response. The status bar at the top right indicates 'Status: 400 Bad Request', 'Time: 140 ms', and 'Size: 522 B'. The JSON response is as follows:

```

{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.1",
  "title": "Validation Errors Occurred.",
  "status": 400,
  "detail": "See the errors property for details",
  "instance": "/api/Employee",
  "Retry": "Please Retry After 30 Minutes",
  "errors": {
    "City": [
      "The City field is required."
    ],
    "Name": [
      "The Name field is required."
    ],
    "Gender": [
      "The Gender field is required."
    ]
  }
}

```

Using ValidationProblemDetails Object in ASP.NET Core Web API

In our previous example, you can see we created a dictionary and then populated it with all the model validation errors. Then, we used the dictionary inside the Extensions property to return the validation error information. Now, the question that should come to your mind is whether we need to set the Model validation errors manually. The answer is No. We can use the `ValidationProblemDetails` instance, which is a subclass of the `ProblemDetails` class.

To include all the model validation errors, we can create a custom `ValidationProblemDetails` object that inherits from `ProblemDetails` and includes them. It automatically populates the `Errors` property with the validation errors from

ModelState. For a better understanding, please modify the Employee Controller as follows:

```
using Microsoft.AspNetCore.Mvc;
using ReturnTypeAndStatusCodes.Models;

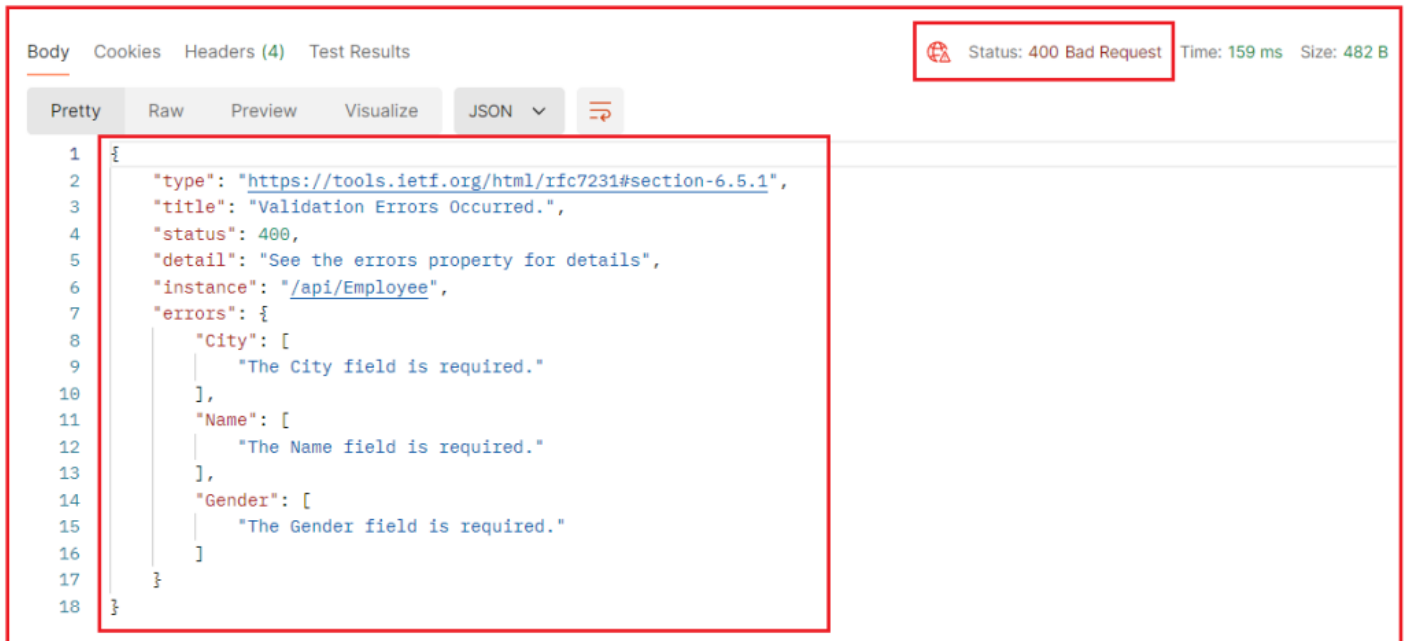
namespace ReturnTypeAndStatusCodes.Controllers
{
    [Route("api/[controller]")]
    // Uncomment this attribute to enable automatic model validation and other
    features
    // [ApiController]
    public class EmployeeController : ControllerBase
    {
        // Define a POST endpoint for creating an employee
        [HttpPost]
        public IActionResult CreateEmployee([FromBody] Employee employee)
        {
            // Check if the model state is valid
            if (!ModelState.IsValid)
            {
                // Returns a 400 Status Code with model state errors
                var problemDetails = new ValidationProblemDetails(ModelState)
                {
                    Title = "Validation Errors Occurred.", // Short description
                    Detail = "See the errors property for details", // Detailed
                    Type = "https://tools.ietf.org/html/rfc7231#section-6.5.1",
                    Status = StatusCodes.Status400BadRequest, // HTTP status code
                    Instance = HttpContext.Request.Path, // The path to the
                };

                // Return a 400 Bad Request response with the problem details
                return BadRequest(problemDetails);
            }

            // Process the request (e.g., save the employee data to the database)
            // ...

            // Return a 200 OK response indicating success
            return Ok();
        }
    }
}
```

Response:



The screenshot shows a web browser's developer tools with the 'Body' tab selected. The response status is '400 Bad Request' with a time of 159 ms and size of 482 B. The response body is a JSON object with the following structure:

```
1 {
2   "type": "https://tools.ietf.org/html/rfc7231#section-6.5.1",
3   "title": "Validation Errors Occurred.",
4   "status": 400,
5   "detail": "See the errors property for details",
6   "instance": "/api/Employee",
7   "errors": {
8     "City": [
9       "The City field is required."
10    ],
11    "Name": [
12      "The Name field is required."
13    ],
14    "Gender": [
15      "The Gender field is required."
16    ]
17  }
18 }
```

Manually Returning 400 HTTP Responses in ASP.NET Core Web API

For more control over the response, you can manually create an `ObjectResult`, specifying the status code directly. For a better understanding, please modify the Employee Controller as follows:

```
using Microsoft.AspNetCore.Mvc;
using ReturnTypesAndStatusCodes.Models;

namespace ReturnTypesAndStatusCodes.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpPost]
        public IActionResult CreateEmployee([FromBody] Employee employee)
        {
            if (!ModelState.IsValid)
            {
                // Manually setting the status code to 400
                Response.StatusCode = 400;

                // You can still return a body
                return new ObjectResult(new { Error = "Invalid data provided" });
            }

            // Process the request...
        }
    }
}
```

```
        // Returns a 200 Status Code indicating success
        return Ok();
    }
}
```

When to Use 400 HTTP Status Code in ASP.NET Core Web API

In ASP.NET Core Web API, understanding when to use 400 Bad Request Status Code can greatly enhance API usability and client error handling. The following are some of the common scenarios where returning a 400 Bad Request is appropriate:

Validation Failures

When the data submitted by the client fails to pass validation checks, either due to missing required fields, incorrect data types, or data that doesn't meet specified constraints, a 400 status code should be returned. This helps the client understand that they need to modify the requested data to comply with the API's expected format.

Missing or Malformed Parameters

If the request lacks required parameters or query strings, or if the format of these parameters is incorrect (e.g., a string where a number is expected), you should return a 400 status code. This also applies to headers or other elements of the HTTP request that are improperly formatted or fail to adhere to the expected schema.

Invalid Request Payload

For scenarios where the request body is not just invalid from a data validation perspective but is syntactically incorrect (malformed JSON or XML, for example), a 400 Bad Request is appropriate. This indicates that the request body must be corrected before the server can properly parse it.

Business Logic Violations

Even if the data is technically correct and well-formed, if it violates business rules or logic (e.g., attempting to create a duplicate resource where uniqueness is required), responding with a 400 can be appropriate. This informs the client that the operation cannot be completed as requested due to semantic errors or rule violations.

In the next article, I will discuss [how to Return the 401 HTTP Status Code in the ASP.NET Core Web API](#) with Examples. In this article, I try to explain **how to Return the 400 Bad Request HTTP Status Code in the ASP.NET Core Web API** with Examples. I hope you enjoy this article on “400 Bad Request HTTP Status Code in the ASP.NET Core Web API.”

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

1 thought on “400 HTTP Status Code in ASP.NET Core Web API”

**OSAMA**

JUNE 29, 2024 AT 2:18 AM

Thank's

[Reply](#)

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

[Consent](#)[Do Not Sell My Information](#)